

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

Aplicaciones Distribuidas (ISR-701) – Unidad 3: Bases de Datos Distribuidas

Análisis Crítico del Modelo de Consistencia y Replicación en el Sistema de Gestión de Solicitudes de Soporte Técnico ISP

Informe Técnico Individual – Código TA-IND-02

Estudiante:	Carlos José Carpio Mendoza
Equipo PFC:	ACC – Soporte Técnico ISP
Rol en el equipo:	Líder de Desarrollo
Asignatura:	Aplicaciones Distribuidas (ISR-701)
Unidad:	3 – Bases de Datos Distribuidas
Docente:	Prof. Guerrero Ulloa G.C.
Periodo:	2026–2027
Fecha:	14 de julio de 2026

Resumen

Este informe analiza si el modelo de consistencia adoptado por el equipo ACC (Soporte Técnico ISP) es óptimo para su dominio: gestión de tickets de un proveedor de Internet. El sistema usa un clúster CockroachDB de tres nodos con fragmentación horizontal por zona y replicación síncrona vía consenso Raft, lo que produce consistencia serializable por defecto. Se ubica este modelo en el espectro de consistencia y se contrasta con dos alternativas: replicación sin líder con quorum ajustable (estilo Dynamo) y replicación multi-líder, mediante una tabla comparativa de cuatro criterios y la lectura del sistema bajo CAP y PACELC. Se argumenta que el modelo es óptimo para operaciones que exigen ausencia de lecturas obsoletas (SLA y facturación), pero impone un costo de disponibilidad ante partición severa que afecta la creación de tickets, operación para la cual el dominio preferiría disponibilidad sobre consistencia estricta. Se concluye que el modelo es casi óptimo, y se proponen mitigaciones a nivel de aplicación para acercar el comportamiento real al ideal deseado sin sacrificar las garantías de facturación.

Palabras clave: consistencia distribuida, CAP, PACELC, replicación, CockroachDB, Raft, bases de datos distribuidas.

1 Introducción

El equipo ACC desarrolla, como Proyecto Fin de Curso (PFC), un Sistema de Gestión de Solicitudes para Soporte Técnico de Internet: una aplicación que permite a los abonados de un proveedor de servicios de Internet (ISP) reportar incidencias, a los técnicos gestionirlas por zona de cobertura, y al negocio monitorear el cumplimiento de acuerdos de nivel de servicio (SLA) y datos de facturación asociados. El dominio combina, dentro de un mismo sistema, operaciones con requisitos de consistencia muy distintos: la creación de un ticket es una operación de alta frecuencia que un abonado espera poder realizar incluso si la infraestructura está parcialmente degradada, mientras que las métricas de SLA y los datos económicos derivados de incumplimientos de contrato no toleran lecturas obsoletas ni escrituras duplicadas.

Esta heterogeneidad de requisitos convierte al PFC en un caso de estudio propicio para la pregunta que estructura este informe: *¿es el modelo de consistencia elegido el óptimo para el dominio del sistema?* Para responderla no basta con describir la tecnología empleada; es necesario ubicar dicho modelo dentro del espectro formal de garantías de consistencia, leerlo a la luz de los teoremas de compromiso CAP y PACELC, y contrastarlo críticamente con alternativas de diseño que el equipo pudo haber tomado y no tomó.

El objetivo de este informe es, por tanto, triple: (i) presentar el fundamento teórico necesario para razonar sobre consistencia y replicación en bases de datos distribuidas (BDD); (ii) caracterizar con precisión el modelo realmente implementado en el PFC del equipo ACC, distinguiendo entre la política *deseada* a nivel de dominio y el comportamiento *real* que ofrece el motor de base de datos elegido; y (iii) evaluar de forma fundamentada si ese modelo es el óptimo, proponiendo alternativas y mitigaciones concretas cuando no lo sea por completo.

2 Marco teórico

2.1 Modelos de consistencia

La consistencia de réplicas en un sistema distribuido no es una propiedad binaria, sino un espectro de garantías con semántica formal precisa, ordenadas por su fuerza relativa [1, 2]. En un extremo

se ubica la **linealizabilidad** (o consistencia fuerte), que exige que toda operación de lectura observe el efecto de la última escritura confirmada según un orden global compatible con el tiempo real en que ocurrieron las operaciones; esto obliga a coordinar cada operación con el resto de las réplicas antes de responder al cliente. Un peldáño por debajo se encuentra la **consistencia secuencial**, que exige un orden total de operaciones compartido por todos los procesos, pero sin la restricción de que ese orden respete el tiempo real; permite ciertas optimizaciones que la linealizabilidad no admite, aunque en la práctica su implementación eficiente es poco frecuente fuera de sistemas de memoria compartida.

La **consistencia causal** relaja aún más la garantía: solo exige que las operaciones causalmente relacionadas (por ejemplo, una lectura que ocurre después de una escritura del mismo proceso, o una escritura que depende de un valor leído previamente) se observen en el mismo orden por todos los nodos, mientras que las operaciones concurrentes e independientes pueden observarse en órdenes distintos. Un caso particular relevante para sistemas orientados a usuario final es la **consistencia de lectura de los propios cambios** (*read-your-writes*), que garantiza únicamente que un proceso siempre observa sus propias escrituras anteriores, sin comprometerse a nada respecto de las escrituras de otros procesos. En el extremo opuesto del espectro se encuentra la **consistencia eventual**, que solo garantiza que, en ausencia de nuevas escrituras, todas las réplicas convergerán eventualmente al mismo valor, sin acotar cuánto tiempo puede tardar esa convergencia ni qué valor intermedio puede observar un lector durante ese lapso [1].

La razón de fondo por la que estos modelos forman un espectro y no una simple lista de opciones es que cada nivel de fuerza semántica exige, a cambio, un costo de coordinación proporcional: un modelo más fuerte, como la linealizabilidad, impone que las réplicas se pongan de acuerdo antes de confirmar cada operación, lo que incrementa la latencia y reduce la disponibilidad ante fallos parciales de la red; un modelo más débil, como la consistencia eventual, permite que cada réplica responda de inmediato con su propio estado local, lo que maximiza la disponibilidad y minimiza la latencia a costa de exponer al cliente datos potencialmente obsoletos o contradictorios entre réplicas [2].

2.2 Teoremas de compromiso: CAP y PACELC

El teorema CAP, formalizado por Gilbert y Lynch a partir de la conjetura original de Brewer, establece que, ante una partición de red (P) que impida la comunicación entre subconjuntos de nodos, un sistema distribuido no puede garantizar simultáneamente Consistencia (C) fuerte y Disponibilidad (A) total: debe elegir entre rechazar operaciones en el lado minoritario de la partición (postura CP) o aceptarlas en todos los lados y reconciliar divergencias después (postura AP) [3].

Sin embargo, CAP describe únicamente el comportamiento del sistema durante una partición, un evento relativamente infrecuente en despliegues bien diseñados. Abadi observa que esta lectura es incompleta, porque incluso en operación normal, sin ninguna partición activa, todo sistema replicado enfrenta un segundo compromiso: entre Latencia (L) y Consistencia (C), formalizado como PACELC (*Partition: Availability or Consistency; Else: Latency or Consistency*) [4]. Un sistema que replica de forma síncrona antes de confirmar cada escritura (posición EC en el eje sin partición) ofrece consistencia fuerte a costa de latencia adicional por cada operación; un sistema que confirma localmente y replica de forma asíncrona (posición EL) minimiza la latencia percibida por el cliente a costa de admitir una ventana de inconsistencia entre réplicas. PACELC es, por tanto, el marco más adecuado para leer decisiones de diseño en sistemas que rara vez sufren particiones severas pero que replican constantemente bajo condiciones normales, como es el caso del clúster del equipo ACC.

2.3 Estrategias de replicación

La estrategia de replicación determina cómo se propagan las escrituras entre réplicas y, con ello, condiciona directamente qué modelo de consistencia resulta alcanzable. En la **replicación primario-respaldo** (*leader-follower*), todas las escrituras de una partición de datos se dirigen a un único nodo líder, que las ordena y las propaga a sus seguidores; esto simplifica el razonamiento sobre el orden de las operaciones, pero concentra la disponibilidad de escritura en un solo nodo por partición. En la **replicación multi-líder**, más de un nodo puede aceptar escrituras de forma concurrente, lo que mejora la disponibilidad de escritura ante particiones geográficas pero introduce el problema de resolver conflictos entre escrituras concurrentes sobre el mismo dato. La **replicación sin líder por cuorum**, popularizada por Dynamo, elimina la figura del líder: el cliente (o un coordinador) escribe y lee de un subconjunto configurable de réplicas, y la consistencia resultante depende de la relación entre el número de réplicas que deben confirmar una escritura (W) y las que deben responder a una lectura (R) frente al total de réplicas (N) [2].

Ortogonalmente a la topología de líderes, la propagación puede ser **síncrona**, donde la operación no se confirma al cliente hasta que un número suficiente de réplicas la ha persistido, o **asíncrona**, donde la confirmación ocurre antes de que todas las réplicas la hayan recibido. Los sistemas de referencia industrial ilustran los extremos de este espacio de diseño: **Dynamo**, de Amazon, prioriza explícitamente la disponibilidad mediante replicación sin líder, cuorums ajustables y consistencia eventual con resolución de conflictos del lado del cliente (*vector clocks*), asumiendo que para su dominio de carrito de compras es preferible aceptar una escritura potencialmente conflictiva que rechazarla [5]; **Spanner**, de Google, ofrece en el otro extremo consistencia externa (transacciones globalmente linealizables) apoyándose en replicación síncrona vía consenso Paxos y en sincronización temporal de alta precisión (TrueTime) para ordenar transacciones distribuidas sin sacrificar disponibilidad práctica [6]. CockroachDB, el motor elegido por el equipo ACC, se ubica conceptualmente más cerca de Spanner que de Dynamo: usa el algoritmo de consenso Raft para replicar de forma síncrona cada rango de datos entre sus réplicas, logrando consistencia serializable sin requerir relojes atómicos especializados [7]. La fragmentación horizontal que acompaña a esta replicación – se detalla en la Sección 3.1 – debe, además, cumplir las tres condiciones formales de corrección de todo esquema de fragmentación: completitud, reconstrucción y disyunción [8].

3 Análisis del modelo de consistencia del PFC

3.1 Caracterización del modelo realmente implementado

El PFC del equipo ACC despliega un clúster CockroachDB de tres nodos (**roach1**, **roach2**, **roach3**), cada uno asociado a una *locality* distinta que corresponde a una zona de cobertura del ISP (quevedo-centro, quevedo-norte, quevedo-sur). Sobre este clúster se aplica una **fragmentación horizontal por lista** (PARTITION BY LIST) sobre la columna **zone** de la tabla **tickets**, con tres particiones nombradas más una partición **DEFAULT** de resguardo, y cada partición se ancla preferentemente, mediante **CONFIGURE ZONE ... constraints**, al nodo cuya localidad coincide con su zona operativa, conservando en todos los casos tres réplicas (una por nodo) para no comprometer la tolerancia a fallos ante la caída de un nodo individual (ver la Figura 3.1 y el esquema del Listado 1 en el repositorio del PFC).

Internamente, CockroachDB replica cada rango de datos (*range*) mediante el algoritmo de consenso **Raft**: toda escritura debe ser aceptada por una mayoría de las réplicas de su rango antes de confirmarse al cliente, lo que produce **replicación síncrona** y aislamiento **serializable** por

defecto en todas las transacciones, sin necesidad de que la aplicación configure explícitamente un nivel de consistencia por tabla, a diferencia de motores multi-modelo que sí exponen ese ajuste [7]. En términos del espectro presentado en la Sección 2.1, el modelo por defecto del PFC se ubica, por tanto, en el extremo más fuerte: **linealizabilidad** / **serializabilidad**, no consistencia eventual.

La fragmentación horizontal por LIST sobre la columna **zone** fue validada por el equipo, en el ADR-0003 del PFC, contra las tres condiciones de corrección que debe satisfacer todo esquema de fragmentación [8]. La **completitud** se cumple porque **zone** es NOT NULL en la tabla **tickets** y la partición **DEFAULT** actúa como resguardo para valores no anticipados, de modo que todo ticket queda asignado a exactamente una partición sin excepción. La **reconstrucción** se cumple porque la unión de las cuatro particiones (**tickets_centro**, **tickets_norte**, **tickets_sur**, **tickets_default**) mediante UNION ALL reproduce exactamente la tabla lógica **tickets**, sin pérdida ni duplicación de filas, propiedad que CockroachDB garantiza de forma transparente al planificador de consultas. La **disyunción** se cumple porque **zone** es un valor discreto y mutuamente excluyente (**QUEVEDO_CENTRO**, **QUEVEDO_NORTE**, **QUEVEDO_SUR**), de modo que ninguna fila puede pertenecer simultáneamente a dos particiones. Que la fragmentación sea formalmente correcta es una condición necesaria, pero no suficiente, para que el modelo de consistencia funcione como se espera: si la disyunción fallara – por ejemplo, por un valor de **zone** inconsistente entre mayúsculas y minúsculas –, una misma entidad de negocio podría quedar dispersa entre dos particiones y, con ello, entre dos grupos Raft distintos, rompiendo implícitamente la linealizabilidad percibida por el **ticket-service** al leer un ticket por su identificador sin conocer de antemano su zona.

3.2 Justificación técnica ligada al dominio

El equipo documentó esta decisión en el ADR-0004 del repositorio del PFC bajo una **postura CAP mixta por agregado de dominio**, en lugar de forzar un único nivel de consistencia global: para el agregado **MetricaSLA** y los datos de facturación se exige una postura **CP**, porque una lectura obsoleta del estado de cumplimiento de un contrato o una escritura duplicada en un registro económico tiene consecuencias contractuales directas; para la creación de tickets (agregado **Ticket**) se declara como objetivo de dominio una postura **AP**, porque un abonado debería poder reportar una incidencia incluso si el clúster está parcialmente degradado.

Leyendo esta decisión bajo PACELC en el eje sin partición (E), el sistema opta de forma consistente por **EC**: prioriza consistencia sobre latencia en toda escritura, porque Raft replica de forma síncrona antes de confirmar el commit, aceptando un costo de latencia adicional de un solo dígito de milisegundos en LAN a cambio de no requerir un proceso de reconciliación posterior. Esta es una decisión razonable para el dominio: a diferencia del carrito de compras de Dynamo, donde una escritura perdida se resuelve fácilmente re-intentando, un ticket de soporte duplicado o corrupto genera trabajo operativo real para un técnico.

Sin embargo, la caracterización **AP** declarada para la creación de tickets no se corresponde exactamente con el comportamiento real del motor: CockroachDB no ofrece un modo AP puro configurable por tabla, como sí lo haría un motor multi-modelo estilo Dynamo o Cassandra. Mientras dos de los tres nodos permanezcan vivos y conectados entre sí, Raft sigue sirviendo escrituras con consistencia serializable (el quorum de mayoría se mantiene), por lo que en la práctica el sistema se comporta como CP incluso para la creación de tickets en el caso normal. La situación AP solo se manifiesta en el caso extremo de una partición de red que aisle a un nodo minoritario: ese nodo dejará de aceptar escrituras hasta reincorporarse al quorum, en lugar de aceptarlas localmente con reconciliación posterior como lo haría un diseño AP genuino. Esta brecha entre la política deseada a nivel de dominio y el comportamiento real del motor elegido es, como se discute en la Sección 5, el punto más débil del modelo actual.

4 Evaluación de alternativas

Para responder con rigor si el modelo Raft/CockroachDB es el óptimo, se contrasta contra dos alternativas de consistencia y replicación plausibles para el mismo dominio.

Alternativa A – Replicación sin líder por cuorum ajustable (estilo Dynamo/Cassandra).

Cada escritura se propaga a N réplicas y se confirma tras recibir respuesta de W de ellas; cada lectura consulta R réplicas. Ajustando $W + R > N$ se recupera consistencia fuerte a nivel de cuorum, o bien, reduciendo W , se prioriza disponibilidad y baja latencia de escritura a costa de consistencia eventual y necesidad de resolución de conflictos (por ejemplo, con *vector clocks* o *last-write-wins*) [5].

Alternativa B – Replicación multi-líder con resolución de conflictos. Más de un nodo (por ejemplo, uno por zona) acepta escrituras localmente de forma concurrente, y los cambios se propagan de forma asíncrona entre líderes, resolviendo conflictos por regla de negocio (por ejemplo, *last-write-wins* por marca de tiempo, o fusión de campos no conflictivos) [2].

La Tabla 1 contrasta el modelo actualmente implementado con estas dos alternativas según cuatro criterios técnicos: consistencia, disponibilidad ante partición, latencia de escritura y complejidad operativa.

Cuadro 1: Comparación del modelo actual del PFC frente a dos alternativas de consistencia y replicación, según cuatro criterios técnicos.

Criterio	Actual: Raft/CockroachDB (líder por rango)	Alt. A: sin líder, cuorum ajustable	Alt. B: multi-líder con conflictos
Consistencia	Serializable/lineal por defecto en todo el clúster	Configurable por operación (W, R); eventual si $W + R \leq N$	Eventual entre líderes; requiere resolución explícita de conflictos
Disponibilidad ante partición	Nodo minoritario aislado deja de aceptar escrituras (CP real)	Todo nodo alcanzable sigue aceptando escrituras (AP genuino)	Cada líder local sigue aceptando escrituras en su zona (AP por zona)
Latencia de escritura	Coordinación Raft síncrona en cada commit (mayor, pero acotada en LAN)	Baja si W pequeño; mayor si se exige cuorum estricto	Baja (confirmación local inmediata)
Complejidad operativa	Baja para el equipo: el motor gestiona consenso y particionamiento de forma nativa	Alta: exige diseñar y probar resolución de conflictos y reparación de lecturas	Alta: exige política de resolución de conflictos y reconciliación entre zonas

La tabla evidencia que ninguna alternativa domina en todos los criterios: la ganancia de disponibilidad de las Alternativas A y B se paga con complejidad operativa adicional – específicamente, la necesidad de diseñar, implementar y probar una política de resolución de conflictos que hoy el equipo ACC no tiene que construir, porque Raft resuelve el orden de las escrituras de forma transparente a la aplicación.

Postura crítica: ¿es el modelo óptimo?

Para el agregado **MetricsSLA** y facturación, el modelo actual es, con alta confianza, el óptimo: ninguna de las dos alternativas es aceptable, porque ambas introducen una ventana de inconsistencia (eventual o de conflicto) que es precisamente lo que este agregado no puede tolerar según los propios requisitos documentados por el equipo. Para la creación de tickets, en cambio, el modelo actual *no* es estrictamente óptimo respecto del objetivo de dominio declarado en el ADR-0004: si el requisito real fuera disponibilidad plena incluso ante una partición severa que aisle un nodo minoritario, la Alternativa A (o una variante híbrida) se ajustaría mejor a ese objetivo específico. Sin embargo, esta suboptimalidad es de bajo impacto práctico: una partición que aisle a un nodo minoritario en un clúster de tres nodos dentro de una misma ciudad (Quevedo) es un evento de baja probabilidad frente al costo de mantenimiento de una política de reconciliación de conflictos permanente. El modelo elegido es, por tanto, **casi óptimo**: correcto para el 90 % de los casos de uso del dominio y deliberadamente conservador en el caso extremo, a cambio de una complejidad operativa significativamente menor para un equipo de cuatro estudiantes con un plazo acotado.

5 Discusión crítica

La pregunta orientadora de este informe – ¿es el modelo de consistencia elegido el óptimo para el dominio del sistema? – admite una respuesta matizada y no un simple sí o no. Bajo el marco PACELC, la decisión de priorizar consistencia sobre latencia (EC) en operación normal es inequívocamente acertada para este dominio: la latencia adicional de un consenso Raft en una red de área local (LAN) es del orden de milisegundos, un costo insignificante frente al beneficio de no tener que diseñar, probar y mantener un mecanismo de reconciliación de conflictos que un equipo estudiantil difícilmente podría validar con el rigor que exige un dato financiero como una métrica de SLA.

Bajo el marco CAP, en cambio, aparece la limitación real identificada en la Sección 3.2: la postura AP declarada como deseable para la creación de tickets no se materializa en el comportamiento del motor ante una partición que aisle a un nodo minoritario. Esta es una limitación genuina, no meramente teórica, porque afecta directamente al abonado que intenta reportar una incidencia justo cuando la infraestructura de red del ISP está fallando – el escenario más probable, en la práctica, para que un abonado necesite reportar un problema. El equipo documentó esta brecha honestamente en el ADR-0004 en lugar de ignorarla, lo cual es en sí mismo una buena práctica de diseño: reconocer explícitamente dónde la herramienta elegida no cubre el ideal teórico permite planificar mitigaciones en lugar de descubrir el problema en producción.

Comportamiento de CockroachDB ante particiones: la matemática del quorum

La brecha identificada arriba no es un defecto de configuración, sino una consecuencia directa de cómo Raft define la disponibilidad de un rango. Cada partición de tickets (`tickets_centro`, `tickets_norte`, `tickets_sur`) es, para efectos de consenso, un grupo Raft independiente con tres réplicas – una por nodo –, cuyo quorum de mayoría es $\lfloor 3/2 \rfloor + 1 = 2$. Mientras dos de las tres réplicas puedan comunicarse entre sí y con el *leaseholder* (la réplica que actúa como líder de lectura/escritura del rango en un momento dado), el grupo Raft continúa eligiendo líder, ordenando el log de operaciones y confirmando escrituras con total normalidad: este es exactamente el escenario que el equipo reproduce en la prueba de tolerancia a fallos del Paso 4 (`docker stop roach2` descrita en el `README.md` de `db-cluster`), donde se espera – y se debe evidenciar en el video de la Entrega 3 – que la carga sostenida de `load_write.py` continúe sin errores porque `roach1` y `roach3` siguen constituyendo mayoría.

El caso cualitativamente distinto, que el PFC no ejerce en su prueba estándar pero que es relevante para esta discusión, es la pérdida simultánea de dos de las tres réplicas de un mismo rango (por ejemplo, una partición de red que aisle a `roach2` y `roach3` y deje a `roach1` en minoría, o la caída simultánea de dos nodos). En ese escenario ninguna de las particiones disponibles alcanza el quorum de 2, el grupo Raft no puede elegir *leaseholder* y toda operación de lectura o escritura dirigida a ese rango bloquea hasta agotar su *timeout* (típicamente expuesta al cliente JDBC como `context deadline exceeded` o como un error de conexión rechazada), en lugar de responder con datos obsoletos. Este es precisamente el mecanismo por el cual CockroachDB nunca se comporta como AP puro ante una partición severa: prefiere *no responder* antes que responder con una escritura que después no pueda reconciliar de forma determinista, lo cual es coherente con su diseño inspirado en Spanner y no en Dynamo [7].

Mitigación propuesta: patrón Outbox adaptado en ticket-service

El propio PFC ya necesita, independientemente de esta discusión de disponibilidad, una solución al clásico *problema de doble escritura*: la coreografía de Sagas documentada en el componente de *messaging* del repositorio establece que `ticket-service` debe publicar un evento `ticket.created` en Kafka cuando un ticket se crea, para que `ai-service` lo consuma, lo clasifique y publique de vuelta `ticket.classified`. Escribir la fila en `tickets` y publicar en Kafka son dos operaciones sobre sistemas distintos que no pueden envolverse en una única transacción distribuida sin sacrificar disponibilidad; el **patrón Outbox** resuelve esto insertando, en la *misma* transacción CockroachDB que crea el ticket, una fila en una tabla `outbox_events` con el *payload* del evento pendiente, y delegando la publicación efectiva a Kafka a un proceso relevador (*poller* o CDC cursor sobre `outbox_events`) que se ejecuta después y marca la fila como publicada; así, la atomicidad la garantiza únicamente CockroachDB, y Kafka recibe el evento con semántica de *al menos una vez*.

Esta arquitectura, sin embargo, presupone que la transacción inicial contra CockroachDB *sí* pudo completarse, precondition que precisamente falla en el escenario de partición severa descrito arriba. Para cerrar esa brecha sin rehacer la capa de persistencia, se propone **extender** el mismo mecanismo hacia el borde de la aplicación, en cuatro pasos concretos:

- (1) El `ticket-service` genera, en el momento de recibir la solicitud HTTP del abonado, un identificador de idempotencia (UUID v4) *en la propia aplicación*, en lugar de delegarlo al `DEFAULT gen_random_uuid()` de la columna `id`; esto es indispensable porque, si la escritura nunca llega a confirmarse en CockroachDB, ningún identificador se habría generado del lado del motor.
- (2) Se intenta la inserción transaccional normal en `tickets` (junto con su fila de `outbox_events`, como ya exige la Saga). Si CockroachDB confirma el commit, el flujo continúa exactamente como en el caso normal: el relevador de `outbox_events` publica `ticket.created` y la clasificación procede sin cambios.
- (3) Si, en cambio, el intento falla con un error retrievable específico de falta de quorum (*timeout* de contexto o rechazo de conexión hacia el rango de la zona correspondiente, distinguible de un error de validación de datos), el `ticket-service` persiste el mismo payload – con su identificador de idempotencia – en una **cola local de contingencia** fuera de CockroachDB (un registro en disco local o, alternativamente, un tópico Kafka dedicado `ticket.pending-write`, dado que Kafka ya forma parte de la infraestructura del PFC y su disponibilidad no depende del clúster CockroachDB), y responde de inmediato al abonado con un acuse de “solicitud recibida, en proceso”, en lugar de propagar el error 503 al usuario final.

- (4) Un proceso de reconciliación en segundo plano reintenta periódicamente, con *backoff* exponencial y *jitter*, la inserción original usando el mismo identificador de idempotencia como clave primaria; al restablecerse el cuorum del rango, la inserción se acepta exactamente una vez (la clave primaria impide duplicados si el intento original había llegado a persistir del lado del clúster pero la confirmación se perdió en la red), momento en el cual el ticket entra recién al flujo normal de la Saga de clasificación.

Esta extensión aproxima, a nivel de aplicación, el comportamiento AP que el motor de base de datos no ofrece de forma nativa para la creación de tickets, sin tocar la garantía CP que se exige para *MetricsSLA* y facturación – esos agregados nunca pasan por la cola de contingencia, precisamente porque para ellos una respuesta inmediata pero potencialmente irreconciliable es peor que ninguna respuesta. Como medida complementaria de observabilidad, se recomienda que las alertas de monitoreo del clúster (consola web de CockroachDB, puerto 8080) incluyan un umbral específico sobre el número de rangos subreplicados, de modo que el equipo de soporte del propio ISP detecte una partición parcial antes de que un volumen significativo de tickets se acumule en la cola de contingencia.

Una segunda limitación, de naturaleza distinta, es la fragmentación por zona documentada en el ADR-0003: las consultas que cruzan zonas (por ejemplo, un reporte global de SLA para todo el ISP) requieren un *scatter-gather* entre las tres particiones, con mayor latencia que una consulta de una sola zona. Esta limitación no compromete la consistencia del sistema – Raft sigue garantizando que cada partición individual responde con datos actualizados –, pero sí su rendimiento para el subconjunto de consultas analíticas que el equipo debe caracterizar empíricamente en el análisis comparativo de la Entrega 3 del PFC.

6 Conclusiones

El modelo de consistencia y replicación adoptado por el equipo ACC – consenso Raft sobre un clúster CockroachDB de tres nodos, con fragmentación horizontal por zona geográfica y replicación síncrona – se ubica en el extremo más fuerte del espectro de consistencia (serializabilidad/linealizabilidad) y corresponde, bajo PACELC, a una postura EC deliberada. Este informe concluye que dicho modelo es **casi óptimo** para el dominio de gestión de tickets de soporte técnico ISP: resulta óptimo sin reservas para los agregados que exigen ausencia de lecturas obsoletas (métricas de SLA, facturación), y resulta subóptimo únicamente en un caso de baja probabilidad – la partición que aísla a un nodo minoritario – para la operación de creación de tickets, donde el dominio preferiría en principio disponibilidad plena.

La comparación con dos alternativas de diseño – replicación sin líder por cuorum ajustable y replicación multi-líder – confirma que ambas habrían resuelto esa brecha puntual a costa de introducir complejidad de resolución de conflictos que el equipo no puede permitirse asumir en el plazo del PFC sin comprometer la integridad de los datos financieros del dominio. La decisión de mantener Raft/CockroachDB como motor único, complementada con una cola de contingencia a nivel de aplicación para el caso extremo, constituye la trayectoria de menor riesgo técnico y académico para el equipo, y se recomienda documentarla explícitamente como tal en la defensa oral de la Entrega 3.

Declaración de uso de IA generativa

En la elaboración de este informe se utilizó la herramienta Claude (Anthropic), a través de Claude Code, con los siguientes propósitos: (i) revisión y organización de la estructura del documento

conforme a la guía de evaluación TA-IND-02; (ii) apoyo en la redacción y corrección de estilo del marco teórico (Sección 2) a partir de las fuentes académicas citadas; (iii) generación del código TikZ de la Figura 3.1 a partir de la descripción textual de la topología del clúster documentada en los ADR-0003 y ADR-0004 del PFC; y (iv) verificación de la resolución correcta del DOI de la referencia [7]. El análisis crítico, la postura frente a la pregunta orientadora, la caracterización del modelo real del PFC y las conclusiones son de autoría y responsabilidad del estudiante, y se fundamentan en el trabajo de diseño previamente realizado y documentado por el equipo ACC en la Entrega 2 y en los ADR de la Entrega 3.

Referencias

- [1] P. Viotti and M. Vukolić, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Computing Surveys*, vol. 49, no. 1, pp. 1–34, 2016, art. no. 19. doi: 10.1145/2926965
- [2] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed. distributed-systems.net, 2017, isbn: 978-1543057386.
- [3] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002. doi: 10.1145/564585.564601
- [4] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, no. 2, pp. 37–42, 2012. doi: 10.1109/MC.2012.33
- [5] G. DeCandia *et al.*, “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP ’07)*, 2007, pp. 205–220. doi: 10.1145/1294261.1294281
- [6] J. C. Corbett *et al.*, “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013, art. no. 8. doi: 10.1145/2491245
- [7] R. Taft *et al.*, “CockroachDB: The Resilient Geo-Distributed SQL Database,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1493–1509. doi: 10.1145/3318464.3386134
- [8] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Springer, 2020, isbn: 978-3-030-26251-8.